

Tackling Sim-to-Real Mismatch in World Models Through A Sampling-Based Disturbance Observer

The International Journal of Robotics Research

XX(X):1–17

©The Author(s) 2026

Reprints and permission:

sagepub.co.uk/journalsPermissions.nav

DOI: 10.1177/ToBeAssigned

www.sagepub.com/



Tianqi Zhu¹, Jun Yang², Jianliang Mao³, Cong Li², and Shihua Li¹

Abstract

Robotic controllers increasingly rely on analytical models, simulators, cost-query interfaces, and learned world models. However, physical deployment can deviate from nominal assumptions, and additional disturbances may arise even when the model itself is accurate. In control systems, disturbance observers (DOB) are widely used to estimate such unmeasured effects from nominal models and measured feedback. Classical DOB formulations are generally built around explicit plant models. This paper develops the sampling-based disturbance observer (SDOB), extending the DOB principle to a broader range of models, including simulators and learned world models, through state-rollout or cost-query interfaces. SDOB separates two observable channels: state-effect disturbances, for which the feedback state differs from its prediction, and cost disturbances, for which the same query state receives different costs as the perceived environment changes. Diverse simulation and real-robot experiments across traditional and learned models demonstrate the effectiveness of SDOB in compensating for sim-to-real mismatch and improving control performance.

Keywords

Sampling-based disturbance observer, sim-to-real, sampling-based model predictive control, world models

1 Introduction

From early model-based control to modern robotics built on simulators and learned models, deployment has always involved discrepancies between simulator models and physical plant. In classical systems, these discrepancies appear as parameter uncertainty, unmodeled dynamics, and external disturbances (Ohnishi et al. 1996; Chen et al. 2016); in contemporary robotics, the same problem is often discussed as the sim-to-real gap (OpenAI et al. 2020; Jaquier et al. 2025). Such discrepancies can degrade tracking performance and closed-loop stability. If a disturbance is measurable, feedforward compensation can often reduce its influence. In practice, the relevant disturbances are often unmeasured or entangled with model uncertainty. Classical DOB formulations estimate these effects in systems described by explicit plant or low-dimensional state-space models (Ohnishi et al. 1996; Chen et al. 2016; Sariyildiz et al. 2020). Conventional MPC operates within the same model-based setting, optimizing behavior over a finite horizon using explicit dynamics and constraints (Garcia et al. 1989; Mayne et al. 2000; Ostafew et al. 2016). DOB estimates can therefore be incorporated into its prediction and optimization

loop, for example by correcting output predictions (Yang et al. 2015).

However, modern robotic systems increasingly use sampling-based MPC when the dynamics or costs are highly nonlinear, nonsmooth, or accessible only through simulators. Sampling-based MPC evaluates candidate action sequences through forward rollouts. Methods such as model predictive path integral control (MPPI) (Williams et al. 2018a; Carius et al. 2022; Koptev et al. 2024) and the cross-entropy method (CEM) (Rubinstein 1997; Kobilarov 2012; Pinneri et al. 2021) rank these candidates using scalar trajectory costs. Because they require only forward

¹School of Automation, Key Laboratory of Measurement and Control of CSE, Ministry of Education, Southeast University, Nanjing 210096, China
²Robotics and Autonomous Systems Thrust, Systems Hub, The Hong Kong University of Science and Technology (Guangzhou), Guangzhou 511453, China

³Faculty of Artificial Intelligence, Shanghai University of Electric Power, Shanghai 200090, China

Corresponding author:

Jun Yang, The Hong Kong University of Science and Technology (Guangzhou), Guangzhou 511453, China; Jianliang Mao, Shanghai University of Electric Power, Shanghai 200090, China.

Email: jy@hkust-gz.edu.cn (Jun Yang); jl_mao@shiep.edu.cn (Jianliang Mao)

rollouts and scalar costs, these methods can use nonlinear simulators without analytic derivatives. At the same time, rollout evaluation has expanded beyond analytical dynamics to perception-derived cost maps and signed-distance fields (Hornung et al. 2013; Oleynikova et al. 2017; Zucker et al. 2013), while world models (Hafner et al. 2019, 2020) learn action-conditioned latent dynamics for prediction and planning. This flexibility allows MPC to use a broader range of models, but it also makes disturbance estimation less direct. The resulting interfaces may provide neither an invertible plant representation nor a clearly defined input-equivalent disturbance channel. Unmodeled effects are therefore revealed through deviations in the feedback state or changes in the costs assigned to queried states, for example after contact, perception updates, or obstacle motion. The controller observes these effects only after executing an action and receiving feedback. Taken together, these changes leave a mismatch between the information required by classical DOB formulations and the interfaces exposed by modern predictive models.

To address this interface mismatch, we present SDOB as a framework for feedback-based disturbance compensation in analytical models, simulators, and learned predictors exposed through rollout or cost-query interfaces. We refer to each member of this family as a modern model. The state channel fits feedback transitions within a task-defined disturbance parameterization, while the cost channel models local transport of environment-dependent costs. Together, these channels specify how observed mismatch enters subsequent rollout and cost evaluation. SDOB therefore targets locally persistent, task-relevant mismatch rather than complete identification of the sim-to-real gap. The main contributions are:

- 1) A new interface-level disturbance representation is introduced for modern models. Unlike conventional DOB formulations that primarily describe disturbances through plant dynamics and state evolution, SDOB separates observable deployment mismatch into two complementary channels: a state-effect disturbance that alters rollout states and a cost disturbance that changes costs at fixed query states. This extends disturbance observation to environment-dependent cost evaluation.
- 2) Based on this representation, SDOB introduces a next-generation approach to disturbance observation that operates through forward rollouts and cost queries. Unlike classical DOB designs built on explicit transfer functions or state-space equations, SDOB does not require access to the analytical expressions

of the underlying model. It estimates both disturbance channels online and incorporates their effects into future state propagation and cost evaluation, without plant inversion or model retraining.

- 3) The framework is extensively validated across diverse simulation environments and real-robot platforms, spanning analytical-model disturbance compensation, moving-obstacle navigation, latent and visual world-model planning, mobile-robot navigation, dual-arm obstacle avoidance, and humanoid dual-arm manipulation with PointWorld, demonstrating its applicability across different model classes and robotic tasks.

The remainder of this article is organized as follows. Section 2 reviews related work. Section 3 introduces the classical DOB formulation used as a reference. Section 4 presents SDOB, including its deployment-oriented interpretation, two disturbance channels, and compensation interfaces. Section 5 reports the simulation and real-robot experiments. Section 6 concludes the article and discusses its limitations and future directions.

2 Related Work

SDOB connects disturbance observation, sampling-based MPC, dynamic environment estimation and cost prediction, sim-to-real adaptation, and world models. Disturbance observation provides the observer principle, sampling-based MPC provides rollout and cost evaluation interfaces, and sim-to-real adaptation motivates correction under deployment mismatch. Dynamic environment estimation and cost prediction provide the context for the cost disturbance channel. World models represent a frontier class within the modern models defined above and provide learned, action-conditioned representations for predicting future states.

2.1 DOB-Based Control

Classical DOB-based control estimates input disturbances from a nominal plant, commonly through an inverse model and low pass filtering, to improve motion control robustness (Ohnishi et al. 1996; Shim and Jo 2009; Sariyildiz et al. 2020; Park and Back 2024). Nonlinear DOB formulations extend this principle to robotic dynamical systems (Chen et al. 2000a,b; Mohammadi et al. 2013; Chen et al. 2016). Active disturbance rejection control (ADRC) and extended state observer (ESO) methods likewise estimate lumped uncertainty in feedback loops (Han 2009; Gao 2003), while DOB-enhanced predictive control incorporates disturbance estimates into prediction and optimization (Yang et al. 2015). Many classical DOB formulations assume an

explicit plant model and a predefined disturbance channel. These assumptions are difficult to retain when control operates on images, latent or object centric states, or rollout distributions available only through forward simulation and cost evaluation.

2.2 Sampling-Based MPC

Sampling-based MPC optimizes candidate action sequences through forward rollouts and ranks, weights, or refits them using trajectory costs (Theodorou et al. 2010; Williams et al. 2015, 2018a; Pinneri et al. 2021). Requiring only simulated futures and scalar costs makes it suitable for nonlinear, nonsmooth, and black box models. Tube-MPPI uses MPPI to generate a nominal trajectory and an ancillary tracking controller to keep the actual state within a tube around that trajectory (Williams et al. 2018b). Robust model predictive path integral control (RMPPI) extends this architecture with augmented rollouts of the nominal and actual dynamics, tracking-aware importance sampling, and safety logic for nominal-state propagation (Gandhi et al. 2021). Parameter uncertainty can be incorporated directly into path integral control (Abraham et al. 2020), while Shield-MPPI integrates control barrier functions for safety (Yin et al. 2023).

2.3 Dynamic Environment Estimation and Cost Prediction

Planning in dynamic environments requires anticipating how environmental motion changes collision risks and trajectory costs. Dynamic occupancy grids estimate occupancy and motion (Nuss et al. 2018), while Occupancy Flow Fields jointly predict their future evolution (Mahjourian et al. 2022). ImplicitO supports continuous occupancy and flow queries at selected locations, avoiding dense scene prediction (Agro et al. 2023). To evaluate candidate trajectories, neural motion planners learn spatiotemporal cost volumes (Zeng et al. 2019), while QuAD queries predicted occupancy at planning-relevant locations (Biswas et al. 2024). These methods primarily focus on predicting occupancy or planning costs, with less emphasis on treating cost changes at fixed query states as a distinct disturbance channel for online estimation and compensation.

2.4 Sim-to-Real Adaptation

Sim-to-real adaptation takes a complementary approach by changing the training distribution, system model, or simulator. Domain randomization broadens the training variation (Tobin et al. 2017), system identification calibrates the dynamics (Tan et al. 2018), and feedback driven

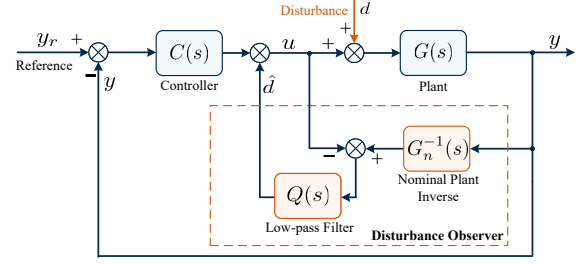


Figure 1. Classical frequency-domain DOB control structure.

simulator adaptation updates the simulator from real experience (Chebotar et al. 2019). Adaptation methods can align latent representations under unknown target appearances (Yoneda et al. 2022), while ManiDreams propagates perceptual, parametric, and structural uncertainty through task specific predictive models (Wang et al. 2026). Such methods adapt the simulator, representation, policy, or predictive state, but remain tied to variations represented during training or adaptation. Greater simulator fidelity also does not guarantee that method rankings or task outcomes transfer to hardware (Kadian et al. 2020), particularly when learned states do not map directly to physical variables.

2.5 World Models for Prediction and Planning

World models learn action conditioned representations for imagined futures, from early latent models (Ha and Schmidhuber 2018) to PlaNet (Hafner et al. 2019), Dreamer (Hafner et al. 2020, 2025), and TD-MPC2 (Hansen et al. 2024). Learned dynamics for manipulation (Ai et al. 2025), broader predictive architectures (LeCun 2022), V-JEPA (Bardes et al. 2024), and PointWorld (Huang et al. 2026) extend this direction toward feature level and 3D action conditioned representations. These models improve prediction, but nominal replanning does not usually preserve recurring execution mismatch as an explicit disturbance. Actuator bias, contact mismatch, object drift, and environmental change can therefore reappear as rollout error at every control cycle.

These developments motivate extending disturbance observation to a broader range of predictive models. To establish the conceptual basis for this extension, the next section revisits the basic principle of classical DOBs.

3 Preliminaries

3.1 Disturbance Observer

Figure 1 shows the classical frequency-domain DOB structure. For clarity, consider a single-input single-output

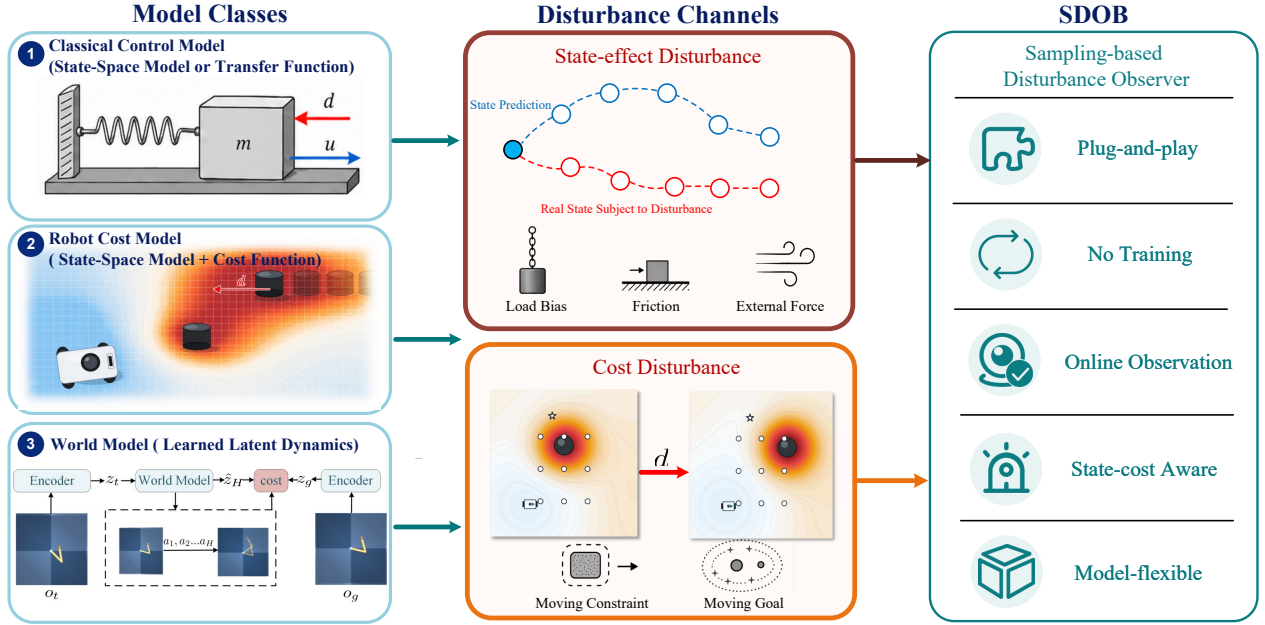


Figure 2. Overview of SDOB: model classes, disturbance channels, and key advantages.

plant with a matched input-side disturbance:

$$y(s) = G(s) (u(s) + d(s)) \quad (1)$$

where $G(s)$ is the actual physical plant, $u(s)$ is the applied control input, and $d(s)$ is an unknown disturbance entering through the same channel as the input. In this formulation, $G_n(s)$ denotes the nominal plant model used for disturbance estimation. The nominal inverse $G_n^{-1}(s)$ maps the measured output back to an input-equivalent quantity, which is then compared with the applied input and filtered before compensation (Ohnishi et al. 1996; Chen et al. 2016; Sariyildiz et al. 2020).

The DOB estimate is

$$\hat{d}(s) = Q(s) (G_n^{-1}(s)y(s) - u(s)), \quad (2)$$

where $Q(s)$ is a stable low-pass filter. The estimate is then used in the input channel as

$$u(s) = u_c(s) - \hat{d}(s), \quad (3)$$

where $u_c(s)$ is the baseline controller output before disturbance compensation.

Example: discrete-time integrator. A minimal discrete-time example is a scalar integrator with a matched disturbance,

$$y_{k+1} = y_k + u_k + d_k, \quad (4)$$

with nominal prediction

$$\hat{y}_{k+1|k} = y_k + u_k. \quad (5)$$

After executing u_k and observing y_{k+1} , the one-step prediction error is

$$e_{k+1} = y_{k+1} - \hat{y}_{k+1|k} = y_{k+1} - y_k - u_k. \quad (6)$$

If the nominal model is exact and measurement noise is ignored, then $e_{k+1} = d_k$. In practice, this residual is passed through a discrete low-pass filter, for example

$$\hat{d}_k = (1 - \alpha)\hat{d}_{k-1} + \alpha e_{k+1}, \quad 0 < \alpha \leq 1, \quad (7)$$

and the next input can be compensated using the previous disturbance estimate,

$$u_k = u_{c,k} - \hat{d}_{k-1}. \quad (8)$$

The observer thus uses the filtered prediction error to compensate for the disturbance in the next control step.

4 Sampling-Based Disturbance Observer

As shown in Figure 2, SDOB extends the classical DOB principle to models accessed through forward rollouts or cost queries. It uses the mismatch between a model query and the following feedback to estimate a correction for the next control step.

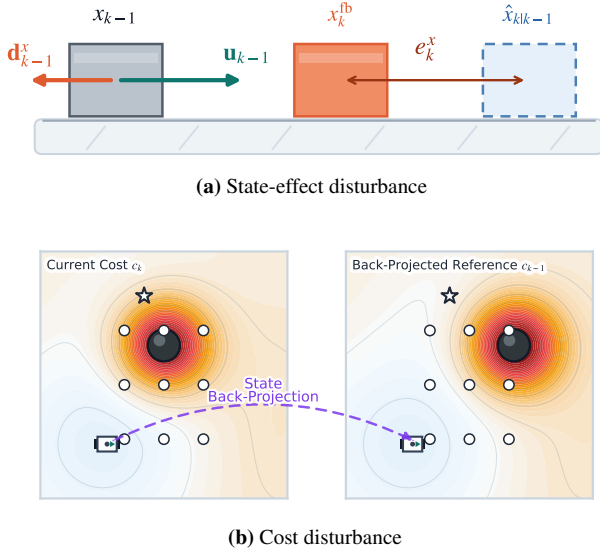


Figure 3. Two disturbance channels used by SDOB. (a) Illustration under the state-effect disturbance d_{k-1}^x . (b) Illustration under the cost disturbance d_{k-1}^c .

4.1 Model and Forward Equations

Let $\mathcal{X}$ denote the known state space, $\mathcal{U}$ the input space, $\mathcal{Q}$ the cost-query space, $\mathcal{E}$ the perceived-environment space, and $\mathcal{D}$ the disturbance space. At the beginning of step k , the controller has the latest feedback state x_k^{fb} , the perceived environment $\mathcal{E}_k$, and the cost-query function c_k . Once the feedback state is passed to the controller, we write $x_k \triangleq x_k^{fb}$. For a query state $q \in \mathcal{Q}$,

$$c_k(q) \triangleq c(q, \mathcal{E}_k). \quad (9)$$

Here, $c_k(q)$ evaluates the query state q in the current perceived environment $\mathcal{E}_k$. The query state collects the information needed for cost evaluation, such as a vehicle pose or joint configuration for collision checking. It is obtained from the known state x through a task-specific mapping:

$$q = \Pi_q(x), \quad \Pi_q : \mathcal{X} \rightarrow \mathcal{Q}. \quad (10)$$

The nominal transition model is $F_x : \mathcal{X} \times \mathcal{U} \rightarrow \mathcal{X}$. The task also supplies a disturbance-enabled rollout interface

$$F_x^d : \mathcal{X} \times \mathcal{U} \times \mathcal{D} \rightarrow \mathcal{X}, \quad F_x^d(x, u, 0) = F_x(x, u). \quad (11)$$

The choice of F_x^d determines the disturbance effects that the state channel can estimate. For example, F_x^d may incorporate an input bias or an additive residual in the predicted state. For each task, we first specify how the disturbance affects the model prediction based on the available knowledge of the system. The interface F_x^d then allows candidate disturbance values to be evaluated through forward prediction. SDOB

compares the predicted transitions with the observed feedback to estimate a disturbance correction for subsequent rollouts. The resulting estimate is therefore a control-equivalent state effect, not necessarily a measured force or a unique physical parameter.

At the beginning of step k , the most recently completed transition is from $k-1$ to k . Its nominal state prediction and nominal cost-query function are

$$\hat{x}_{k|k-1} = F_x(x_{k-1}, u_{k-1}), \quad (12)$$

$$\hat{c}_{k|k-1}(q) = c_{k-1}(q) = c(q, \mathcal{E}_{k-1}). \quad (13)$$

After u_{k-1} is executed, the feedback process returns the current state x_k^{fb} and perceived environment $\mathcal{E}_k$. With an unknown state-effect disturbance, the completed transition and the current cost query can be written as

$$\begin{aligned} x_k^{fb} &= F_x^d(x_{k-1}, u_{k-1}, d_{k-1}^x), \\ c_k(q) &= c(q, \mathcal{E}_k). \end{aligned} \quad (14)$$

Thus $\hat{x}_{k|k-1}$ and $\hat{c}_{k|k-1}$ are formed before execution, whereas x_k^{fb} , $\mathcal{E}_k$, and $c_k(q)$ become available afterward. Here d_{k-1}^x denotes the unknown state-side effect acting over the completed interval.

4.2 Two Disturbance Channels

SDOB separates mismatch by where it appears. A state-effect disturbance changes the feedback state, for example through load, friction, contact, or actuator bias. A cost disturbance changes the cost assigned to the same query state as the perceived environment changes. This second channel is useful when a point cloud, occupancy layer, or collision model is available for queries but is not maintained as a compact dynamic state. As illustrated in Figure 3,

$$d_{k-1} = (d_{k-1}^x, d_{k-1}^c(\cdot)) \quad (15)$$

where d_{k-1}^x denotes a state-effect disturbance and $d_{k-1}^c(q)$ denotes the cost disturbance at query state q . In this work, $d_{k-1}^c(\cdot)$ is a query-dependent vector field that describes the apparent local transport rate of the queried cost, rather than the physical velocity of an individual object.

Remark 1. SDOB requires three conditions. First, the controller must provide a repeatable rollout interface or comparable cost queries across consecutive updates. Second, the selected disturbance coordinates must affect the available feedback or queried costs; components that do not affect these signals are unobservable. Third, the disturbances are assumed to vary slowly over successive control updates,

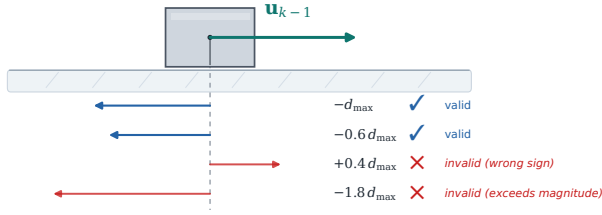


Figure 4. Physical admissibility of sampled state-effect disturbances.

allowing their current estimates to be used for short-horizon prediction. The state channel additionally requires the disturbance-enabled interface in Eq. (11). The cost channel assumes that local cost changes over one update can be approximated by short-horizon transport. Abrupt topology changes or several unresolved motions in the same neighborhood can violate this assumption.

4.3 State-Effect Disturbance Estimation

At the beginning of step k , the controller has received $\mathbf{x}_k^{\text{fb}}$ after executing $\mathbf{u}_{k-1}$. Let $\hat{\mathbf{d}}_{k-1}^x$ denote a candidate estimate of the unknown disturbance $\mathbf{d}_{k-1}^x$ that acted during this completed transition. The candidate estimate produces the prediction

$$\hat{\mathbf{x}}_{k|k-1}(\hat{\mathbf{d}}_{k-1}^x) = F_x^d(\mathbf{x}_{k-1}, \mathbf{u}_{k-1}, \hat{\mathbf{d}}_{k-1}^x). \quad (16)$$

The corresponding state-effect residual is the difference between the feedback state and this prediction

$$\mathbf{e}_k^x(\hat{\mathbf{d}}_{k-1}^x) = \mathbf{x}_k^{\text{fb}} - \hat{\mathbf{x}}_{k|k-1}(\hat{\mathbf{d}}_{k-1}^x). \quad (17)$$

The residual in Eq. (17) scores how well each candidate disturbance explains the completed feedback transition. The disturbance estimate is obtained by minimizing

$$\begin{aligned} \hat{\mathbf{d}}_{k-1}^{x,*} &\in \arg \min_{\hat{\mathbf{d}}_{k-1}^x \in \mathcal{D}} \mathcal{L}_{k-1}^x(\hat{\mathbf{d}}_{k-1}^x), \\ \mathcal{L}_{k-1}^x(\hat{\mathbf{d}}_{k-1}^x) &= \underbrace{\ell_{\text{pred},k-1}^x(\hat{\mathbf{d}}_{k-1}^x)}_{\text{current prediction error}} + \underbrace{\ell_{\text{reg},k-1}^x(\hat{\mathbf{d}}_{k-1}^x)}_{\text{disturbance regularization}} \\ &\quad + \underbrace{\ell_{\text{adm},k-1}^x(\hat{\mathbf{d}}_{k-1}^x)}_{\text{physical admissibility}} \\ \ell_{\text{pred},k-1}^x(\hat{\mathbf{d}}_{k-1}^x) &= \left\| W_x \mathbf{e}_k^x(\hat{\mathbf{d}}_{k-1}^x) \right\|^2, \\ \ell_{\text{reg},k-1}^x(\hat{\mathbf{d}}_{k-1}^x) &= \left\| W_d^x \hat{\mathbf{d}}_{k-1}^x \right\|_2^2, \\ \ell_{\text{adm},k-1}^x(\hat{\mathbf{d}}_{k-1}^x) &= R_{\text{adm}}^x(\hat{\mathbf{d}}_{k-1}^x, \mathbf{x}_{k-1}, \mathbf{u}_{k-1}, \mathbf{x}_k^{\text{fb}}). \end{aligned} \quad (18)$$

The prediction term measures the feedback mismatch, with W_x setting the relative importance of the feedback coordinates. The regularization term discourages unnecessarily large disturbance estimates, with W_d^x weighting the disturbance coordinates. The penalty function R_{adm}^x penalizes violations of known physical constraints, as illustrated in Figure 4. For example, a candidate disturbance exceeding a known magnitude bound incurs a penalty.

When Eq. (18) can be solved analytically, the disturbance is estimated directly. For sampled or black-box rollout models, SDOB instead draws m_x candidate disturbances for the completed transition,

$$\hat{\mathbf{d}}_{k-1,r}^x \sim \rho_{k-1}^x, \quad r = 1, \dots, m_x. \quad (19)$$

Here ρ_{k-1}^x denotes the proposal probability distribution used to sample candidate state-effect disturbances. It may be centered on the previous disturbance estimate. Each candidate is inserted into the loss in Eq. (18). Candidates with lower losses receive larger exponential weights:

$$\ell_{k-1,r}^x = \mathcal{L}_{k-1}^x(\hat{\mathbf{d}}_{k-1,r}^x), \quad (20)$$

$$\omega_{k-1,r}^x = \frac{\exp(-\ell_{k-1,r}^x / \lambda_d)}{\sum_{q=1}^{m_x} \exp(-\ell_{k-1,q}^x / \lambda_d)}. \quad (21)$$

The normalized exponential weights define an empirical Gibbs distribution (Theodorou et al. 2010; Williams et al. 2018a), with $\lambda_d > 0$ controlling its concentration. The point estimate used in the experiments is its weighted mean,

$$\hat{\mathbf{d}}_{k-1}^x = \sum_{r=1}^{m_x} \omega_{k-1,r}^x \hat{\mathbf{d}}_{k-1,r}^x. \quad (22)$$

The weighted mean provides a practical point estimate by combining the sampled disturbances according to their losses, without additional model evaluations once the candidates have been scored. This estimate need not coincide with a minimizer of Eq. (18). The same fitting objective could also be addressed by selecting the candidate with the lowest loss or by iteratively refitting the sampling distribution to an elite set using the CEM (Kobilarov 2012; Pinneri et al. 2021). These are alternative sampling-based solution strategies. For simplicity of presentation, we use the weighted mean throughout this paper for sampled state-effect estimation. The estimate becomes available at step k , remains constant over the next prediction horizon, and is computed using only forward rollouts.

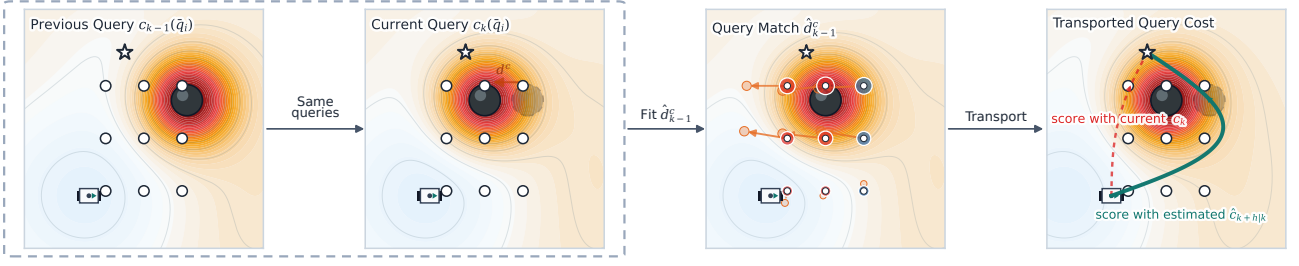


Figure 5. Cost disturbance estimation and planning compensation. The first two panels compare costs at the same query locations across consecutive updates. The third panel estimates local cost motion by testing shifts of the query locations in the previous cost field. Each shift is evaluated by how well the surrounding costs match those in the current field, and the best match determines the local motion estimate. The last panel uses the estimated motion to predict future costs and adjust the planned route to account for the changing environment.

4.4 Cost Disturbance Estimation

The cost channel estimates how environmental changes affect planning costs while the task objective remains fixed. Following the design intuition of compression as intelligence, it uses local cost queries to retain task-relevant information without reconstructing the full environment or tracking every object. For example, a moving obstacle shifts the surrounding high-cost region. SDOB estimates this local motion of the cost field from queries before and after an environment update and uses it to predict future costs, without requiring an explicit model of obstacle motion.

We denote the local velocity of the cost field by $d^c(q, t)$ and refer to it as the cost disturbance. Its admissible values form $\mathcal{D}_c$. This velocity describes motion in query space and need not equal the physical velocity of an individual obstacle.

For a differentiable cost field, the relation between its local motion and its change over time is

$$\frac{\partial c(q, t)}{\partial t} + [d^c(q, t)]^\top \nabla_q c(q, t) = s^c(q, t). \quad (23)$$

The source term $s^c(q, t)$ accounts for cost changes that the local motion does not explain, such as a newly appearing obstacle. When this term is negligible, the cost remains approximately unchanged along the motion of the field, following the same idea as brightness constancy in optical flow (Horn and Schunck 1981).

SDOB assumes that this approximation holds between consecutive cost updates. Let Δt be the time between these updates. The displacement over $[k-1, k]$ is approximately $\Delta t d_{k-1}^c(q)$, giving

$$c_k(q) \approx c_{k-1}(q - \Delta t d_{k-1}^c(q)). \quad (24)$$

In other words, the current cost at q is matched to the previous cost at a position shifted backward by the estimated displacement. SDOB evaluates this relation directly through cost queries, without computing cost gradients. It can

therefore also be applied to nonsmooth costs and binary collision queries. Changes that cannot be explained by the shift remain in the matching residual.

To estimate the local motion, SDOB selects N_c query states, $\mathcal{Q}_k^c = \{q_k^{(i)}\}_{i=1}^{N_c}$, at step k . Figure 5 illustrates the process with $N_c = 9$. A cost change at one point is generally insufficient to determine a motion vector. Each candidate velocity $\hat{d}_{k-1,i}^c$ is therefore tested over a neighborhood $\mathcal{N}_i$ around query state $q_k^{(i)}$, assuming approximately uniform motion within that neighborhood. For each neighbor j , the candidate shifts the query position in the previous cost field to predict the current cost:

$$\hat{c}_{k|k-1}^{c,(i,j)}(\hat{d}_{k-1,i}^c) = c_{k-1}(q_k^{(j)} - \Delta t \hat{d}_{k-1,i}^c), \quad (25)$$

The difference between the current queried cost and this prediction is

$$e_{k,i,j}^c(\hat{d}_{k-1,i}^c) = c_k(q_k^{(j)}) - \hat{c}_{k|k-1}^{c,(i,j)}(\hat{d}_{k-1,i}^c). \quad (26)$$

The local estimate minimizes the weighted matching error across the neighborhood, together with a penalty on the velocity magnitude:

$$\begin{aligned} \hat{d}_{k-1,i}^{c,*} &\in \arg \min_{\hat{d}_{k-1,i}^c \in \mathcal{D}_c} \mathcal{L}_{k-1,i}^c(\hat{d}_{k-1,i}^c), \quad (27) \\ \mathcal{L}_{k-1,i}^c(\hat{d}_{k-1,i}^c) &= \underbrace{\sum_{j \in \mathcal{N}_i} W_{ij}^c [e_{k,i,j}^c(\hat{d}_{k-1,i}^c)]^2}_{\text{local cost matching}} \\ &\quad + \underbrace{\|W_d^c \hat{d}_{k-1,i}^c\|_2^2}_{\text{transport regularization}}. \end{aligned}$$

The nonnegative weights W_{ij}^c set each neighbor's contribution and sum to one over $\mathcal{N}_i$. The matrix W_d^c controls the penalty on large velocities. SDOB tests a finite set of sampled or gridded velocities in $\mathcal{D}_c$ and retains the candidate with the lowest loss at each query state.

Rollouts may query positions outside the selected set. SDOB interpolates the cost-field velocity at these positions by averaging nearby estimates, weighted by distance and confidence. The resulting kernel interpolation (Nadaraya 1964; Kang and Joseph 2016) is

$$\begin{aligned} Z_c(\mathbf{q}) &= \sum_{j=1}^{N_c} W_c^{(j)} K(\|\mathbf{q} - \mathbf{q}_k^{(j)}\|/\sigma_c), \\ \alpha_i(\mathbf{q}) &= \begin{cases} \frac{W_c^{(i)} K(\|\mathbf{q} - \mathbf{q}_k^{(i)}\|/\sigma_c)}{Z_c(\mathbf{q})}, & Z_c(\mathbf{q}) > 0, \\ 0, & Z_c(\mathbf{q}) = 0, \end{cases} \\ \hat{\mathbf{d}}_{k-1}^c(\mathbf{q}) &= \sum_{i=1}^{N_c} \alpha_i(\mathbf{q}) \hat{\mathbf{d}}_{k-1,i}^c. \end{aligned} \quad (28)$$

Here $W_c^{(i)} \geq 0$ is the confidence of estimate i , and K is a nonnegative kernel with spatial scale $\sigma_c > 0$. The factor $Z_c(\mathbf{q})$ normalizes the weights. If no local estimate contributes, $Z_c(\mathbf{q}) = 0$ and the velocity at $\mathbf{q}$ is set to zero.

Under the slowly time-varying disturbance assumption, SDOB holds the estimated velocity field fixed over the prediction horizon. To predict the cost at $\mathbf{q}$ after h steps, it traces that position backward through the estimated field for h steps, then queries the current cost at the resulting position. Starting from $\mathbf{q}_h^{[0]} = \mathbf{q}$, the backward steps are

$$\mathbf{q}_h^{[r+1]} = \mathbf{q}_h^{[r]} - \Delta t \hat{\mathbf{d}}_{k-1}^c(\mathbf{q}_h^{[r]}), \quad r = 0, \dots, h-1. \quad (29)$$

The predicted cost is then

$$\hat{c}_{k+h|k}^c(\mathbf{q}) = c_k(\mathbf{q}_h^{[h]}), \quad h = 1, \dots, H. \quad (30)$$

The minus sign in Eq. (29) traces $\mathbf{q}$ backward along the estimated motion to the corresponding location in the current cost field. Equation (30) then uses the cost at this location to predict the cost of future rollouts.

Remark 2. Local cost queries and limitations. Cost motion cannot be identified in constant-cost regions and is ambiguous along straight obstacle boundaries. Multiple local motions or obstacle appearance and disappearance may violate the transport assumption, while incomplete observations near sensing or map boundaries can introduce spurious cost changes. In practice, restricting $\mathbf{q}$ to a region around the robot or its predicted trajectory helps keep query neighborhoods and shifted positions within the reliably observed area. This reduces errors near sensing or map boundaries without resolving local motion ambiguities.

4.5 Using SDOB in Control

Figure. 6 and Algorithm 1 summarize how SDOB enters the control update. At step k , $\hat{\mathbf{d}}_{k-1}^x$ compensates the state rollouts, while the cost disturbance constructs the horizon query function $\hat{c}_{k+h|k}^c$ through Eqs. (29) and (30). Let

$$U_k = (\mathbf{u}_{k|k}, \dots, \mathbf{u}_{k+H-1|k}) \quad (31)$$

be a horizon control sequence. The controller solves

$$\begin{aligned} U_k^* &= \arg \min_{U_k} S_k(U_k), \\ S_k(U_k) &= \underbrace{\Phi_{\text{cost},k}(U_k)}_{\text{cost channel term}} + \underbrace{\Phi_{\text{other},k}(U_k)}_{\text{other constraint cost}} + \\ &\quad \underbrace{\Phi_{g,k}(U_k)}_{\text{goal cost}} + \underbrace{\Phi_{u,k}(U_k)}_{\text{u smooth}}, \\ \Phi_{\text{cost},k}(U_k) &= \sum_{h=1}^H \hat{c}_{k+h|k}^c(\mathbf{q}_{k+h|k}), \\ \Phi_{\text{other},k}(U_k) &= \sum_{h=0}^{H-1} \ell_k^{\text{con}}(\mathbf{x}_{k+h|k}, \mathbf{u}_{k+h|k}), \\ \Phi_{g,k}(U_k) &= F_c^f(\mathbf{x}_{k+H|k}, \mathbf{x}_g), \\ \Phi_{u,k}(U_k) &= \sum_{h=0}^{H-1} \ell_k^{u,\text{sm}}(\Delta \mathbf{u}_{k+h|k}), \\ \text{s.t. } \mathbf{x}_{k|k} &= \mathbf{x}_k^{\text{fb}}, \\ \mathbf{x}_{k+h+1|k} &= F_x^d(\mathbf{x}_{k+h|k}, \mathbf{u}_{k+h|k}, \hat{\mathbf{d}}_{k-1}^x). \end{aligned} \quad (32)$$

The objective contains four terms. The cost channel term $\Phi_{\text{cost},k}$ evaluates costs exposed through the cost-query interface, such as collision cost, at each predicted task query state using the transported query function. The term $\Phi_{\text{other},k}$ collects constraints not handled through this interface, such as state or input limits, and is not modified by cost disturbance compensation. The goal term $\Phi_{g,k}$ measures the predicted goal error, while $\Phi_{u,k}$ penalizes changes between adjacent inputs to promote smooth control commands.

5 Simulation and Experiments

We evaluate SDOB through five simulation studies and three robot studies spanning analytical and learned prediction models. The comparisons assess the task-level performance of complete controller configurations incorporating SDOB, with task-specific disturbance representations and prediction interfaces. Each study addresses a particular use of feedback correction, and its conclusions are interpreted within the corresponding task conditions and execution protocol.

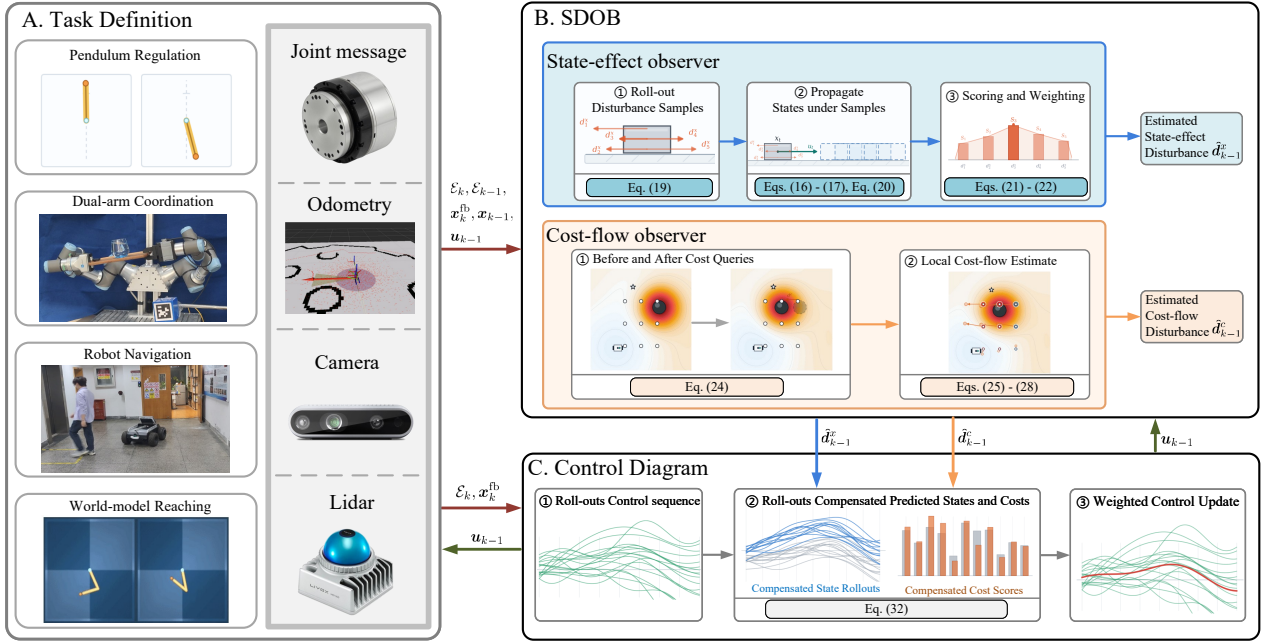


Figure 6. Schematic of the SDOB computation flow. Feedback updates the state-effect and cost disturbance estimates for subsequent rollout and cost compensation.

Algorithm 1: SDOB control update

Input: Transition $(\mathbf{x}_{k-1}, \mathbf{u}_{k-1}, \mathbf{x}_k^{\text{fb}})$; queries $c_{k-1}, c_k; F_x^d$, Π_q , and H
Output: $\hat{\mathbf{d}}_{k-1}^x, \hat{\mathbf{d}}_{k-1}^c(\cdot)$, and $\mathbf{u}_k$
 // State-effect estimation
 1 Sample $\hat{\mathbf{x}}_{k-1,r}^x \sim \rho_{k-1}^x, r = 1, \dots, m_x$;
 2 **for** $r = 1, \dots, m_x$ **do**
 3 Predict $\hat{\mathbf{x}}_{k|k-1}^{(r)} \leftarrow F_x^d(\mathbf{x}_{k-1}, \mathbf{u}_{k-1}, \hat{\mathbf{x}}_{k-1,r}^x)$;
 4 Compute $\mathbf{e}_k^{x,(r)} \leftarrow \mathbf{x}_k^{\text{fb}} - \hat{\mathbf{x}}_{k|k-1}^{(r)}$;
 5 $\ell_{k-1,r}^x \leftarrow \mathcal{L}_{k-1}^x(\hat{\mathbf{x}}_{k-1,r}^x)$ via (18);
 6 Normalize weights $\omega_{k-1,r}^x$ via (21);
 7 Set $\hat{\mathbf{d}}_{k-1}^x \leftarrow \sum_{r=1}^{m_x} \omega_{k-1,r}^x \hat{\mathbf{x}}_{k-1,r}^x$;
 // Cost disturbance estimation
 8 Select $\mathcal{Q}_k^c$ and neighborhoods $\mathcal{N}_i$;
 9 **for** $i = 1, \dots, N_c$ **do**
 10 Evaluate candidate cost residuals over $\mathcal{N}_i$ via (26);
 11 Select the minimum-loss velocity $\hat{\mathbf{d}}_{k-1,i}^c$ via (27);
 12 Interpolate $\hat{\mathbf{d}}_{k-1}^c(\cdot)$ using (28);
 // Compensated control
 13 **foreach** candidate control sequence U_k **do**
 14 Initialize $\mathbf{x}_{k|k} \leftarrow \mathbf{x}_k^{\text{fb}}$;
 15 **for** $h = 0, \dots, H-1$ **do**
 16 Predict $\mathbf{x}_{k+h+1|k} \leftarrow F_x^d(\mathbf{x}_{k+h|k}, \mathbf{u}_{k+h|k}, \hat{\mathbf{d}}_{k-1}^x)$;
 17 Extract $\mathbf{q}_{k+h+1|k} \leftarrow \Pi_q(\mathbf{x}_{k+h+1|k})$;
 18 Backtrace $\mathbf{q}_{k+h+1|k}$ and query its cost via (29)–(30);
 19 Evaluate $S_k(U_k)$ via (32);
 20 Solve (32) for U_k^* ;
 21 Apply $\mathbf{u}_k = \mathbf{u}_{k|k}^*$;

Within each simulation task, methods share the nominal model, nominal task objective, action limits, initial conditions, evaluation seeds, prediction horizon, and

nominal rollout budget. Observer, robustness, and cost-correction mechanisms form part of the evaluated controller configurations; their effects on rollout and cost evaluation are described in the task-specific setups below. Repeated runs use paired task or planner seeds, and method-specific settings remain fixed. SDOB is compared with the corresponding nominal controller and, when available, DOB, Tube-MPPI, and RMPPI. Reported planning time measures wall-clock computation and includes controller-specific observer and robustness computations. The simulation tasks and the real-robot controller computations use an NVIDIA GeForce RTX 4080 Laptop GPU. Table 1 lists the core simulation settings; the remaining observer and controller parameters are provided in the open-source configuration files.

The analytical-model studies examine state-effect compensation and navigation under combined state and cost disturbances. The learned-model studies examine recovery from an imposed action bias, correction of environment-dependent costs, and initial feedback calibration for planning with a frozen predictor.

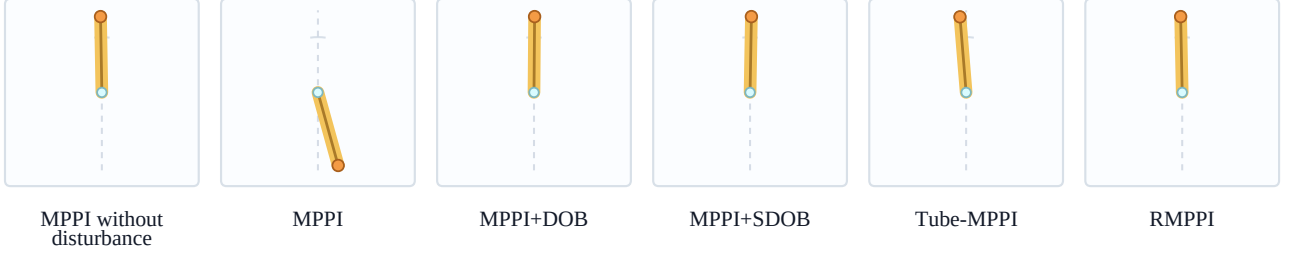
Closed-loop task performance is assessed using success rate together with task-specific tracking and safety metrics. Command smoothness is quantified by control total variation (TV)

$$\text{TV}(\mathbf{u}) = \frac{1}{T-1} \sum_{k=0}^{T-2} \|\mathbf{u}_{k+1} - \mathbf{u}_k\|_2 \quad (33)$$

where a lower value indicates smaller changes between consecutive commands. Computation time is summarized by

Table 1. Core SDOB and controller parameters used in simulation.

Task	SDOB channel	SDOB state samples m_x	SDOB query states N_c	Controller rollout samples	Prediction horizon H	Environment step
Inverted pendulum	State effect	256	–	1024	15	0.050 s
Eight-obstacle navigation	State and cost	256	332	500	30	0.050 s
Latent Reacher	State effect	256	–	300	5	0.020 s
Rocket Collect	Cost	–	289	32	12	0.033 s
Landing	State and cost	256	121	32	24	0.033 s

**Figure 7.** Representative inverted-pendulum snapshots under sinusoidal torque bias. MPPI+SDOB effectively compensates for the torque disturbance, achieving disturbance-rejection performance comparable to the conventional MPPI+DOB.**Table 2.** Inverted-pendulum simulation results.

Method	Success	Tail $ \theta $ (rad)	Settling time (s)	Disturbance RMSE	Control TV	P95 planning time (ms)
MPPI (no disturbance)	20/20	0.008 ± 0.004	0.897 ± 0.203	–	0.125 ± 0.009	8.274 ± 0.541
MPPI	0/20	1.108 ± 0.006	–	–	0.172 ± 0.010	8.024 ± 0.462
MPPI+DOB	20/20	0.027 ± 0.004	2.413 ± 0.142	0.018 ± 0.001	0.127 ± 0.007	8.018 ± 0.405
MPPI+SDOB	20/20	0.026 ± 0.004	2.402 ± 0.157	0.014 ± 0.001	0.127 ± 0.007	8.232 ± 0.426
Tube-MPPI	19/20	0.081 ± 0.008	2.668 ± 0.662	–	0.162 ± 0.007	16.914 ± 1.192
RMPPI	20/20	0.021 ± 0.006	2.423 ± 0.227	–	0.178 ± 0.010	19.628 ± 1.143

the 95th percentile (P95) of per-step planning time. Tabulated values with $\pm$ report the mean and standard deviation across trials. Success and collision entries report trial counts.

5.1 Simulation Results

1) *Inverted-pendulum state-effect task*: This study evaluates state-effect compensation with an analytical nominal model and compares SDOB with a conventional DOB and robust MPPI variants. The task is an inverted pendulum subject to a sinusoidal torque disturbance. Figure 7 shows representative snapshots under the common disturbance, and Table 2 summarizes 20 seeds.

Except for the disturbance-free nominal reference, all methods are evaluated under the same sinusoidal torque disturbance $d(t) = 1.2 \sin(1.2t)$ N m. Success requires $|\theta| < 0.12$ rad for at least the final 20% of the rollout. The uncompensated MPPI controller fails in all 20 trials, whereas MPPI+DOB, MPPI+SDOB, and RMPPI succeed in all trials and Tube-MPPI succeeds in 19. MPPI+DOB and MPPI+SDOB have similar tail error and control variation, but MPPI+SDOB reduces the disturbance estimation RMSE from 0.018 to 0.014. RMPPI achieves a smaller tail error at more than twice the P95 planning time of MPPI+SDOB.

2) *Eight-obstacle navigation with state-effect and cost disturbances*: This study evaluates the complete SDOB configuration in an eight-obstacle navigation task where both state and cost disturbances affect planning. A constant input bias changes the robot motion, while moving obstacles change the queried costs. The obstacles approach from several directions and may overlap. All controllers use the same obstacle margin. Figure 8 shows six instants, and Table 3 reports 20 seeds.

Each run has a 30 s limit, and success means reaching the 0.35 m goal region before collision. The path ratio is the traveled path length divided by the straight-line distance from the start to the goal, so a lower value indicates a more direct route. Goal time and path ratio use successful trials only; negative clearance denotes overlap. MPPI, MPPI+DOB, and Tube-MPPI collide in all trials, whereas MPPI+SDOB and RMPPI succeed in all trials. MPPI+SDOB has a shorter mean path ratio than RMPPI with similar clearance, while its local query update gives the largest planning time.

3) *Latent world-model Reacher task*: This experiment examines whether SDOB can use prediction errors in the model’s latent space to compensate for an imposed action bias and recover performance on tasks already solvable by

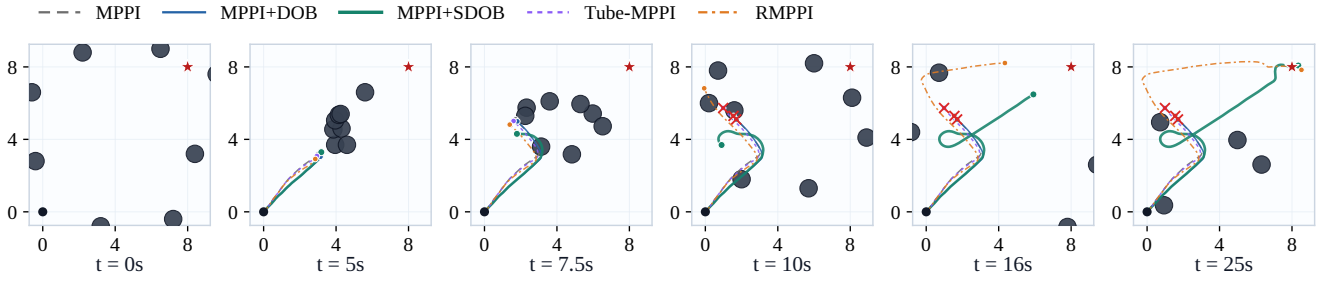


Figure 8. Snapshots comparing navigation methods in an eight-obstacle environment with state-effect and cost disturbances. MPPI+SDOB successfully avoids collisions while achieving a shorter navigation path than the comparison methods.

Table 3. Eight-obstacle navigation simulation results.

Method	Success	Collision	Min. clearance (m)	Goal time (s)	Path ratio	Control TV	P95 planning time (ms)
MPPI	0/20	20/20	-0.006 ± 0.004	—	—	0.697 ± 0.053	7.368 ± 0.708
MPPI+DOB	0/20	20/20	-0.009 ± 0.004	—	—	0.689 ± 0.051	8.736 ± 1.410
MPPI+SDOB	20/20	0/20	0.129 ± 0.034	21.323 ± 2.846	1.484 ± 0.171	0.554 ± 0.034	20.770 ± 1.661
Tube-MPPI	0/20	20/20	-0.002 ± 0.002	—	—	0.674 ± 0.046	7.771 ± 0.889
RMPPI	20/20	0/20	0.125 ± 0.010	21.225 ± 0.026	1.573 ± 0.003	0.518 ± 0.025	18.764 ± 1.146

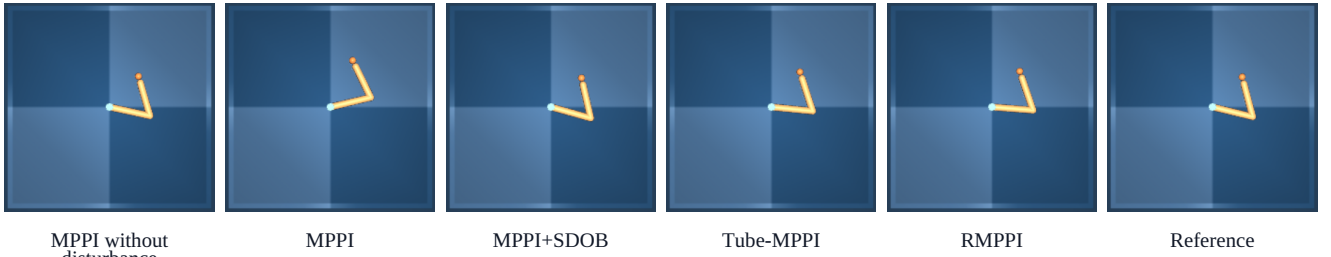


Figure 9. Final Reacher observations from representative evaluations, where MPPI+SDOB achieves a smaller reaching error.

Table 4. Reacher simulation results.

Method	Success	Tail mean error (rad)	Final error (rad)	Disturbance RMSE	Control TV	P95 planning time (ms)
MPPI (no disturbance)	20/20	0.017 ± 0.004	0.016 ± 0.007	—	0.104 ± 0.011	438.956 ± 41.523
MPPI	0/20	0.427 ± 0.038	0.431 ± 0.038	—	0.125 ± 0.016	456.264 ± 69.320
MPPI+SDOB	19/20	0.030 ± 0.010	0.031 ± 0.012	0.056 ± 0.011	0.126 ± 0.020	502.569 ± 5.313
Tube-MPPI	0/20	0.185 ± 0.062	0.220 ± 0.105	—	0.163 ± 0.051	492.350 ± 38.858
RMPPI	0/20	0.176 ± 0.044	0.208 ± 0.109	—	0.152 ± 0.043	492.228 ± 41.545

the frozen model and nominal planner. The second state-effect study uses the pretrained Reacher checkpoint released with LeWorldModel (LeWM) (Maes et al. 2026). The controller encodes current and goal images, samples actions with MPPI, and scores latent rollouts. A constant action bias $[0.30, -0.20]$ is applied during execution but hidden from the planner. SDOB estimates an equivalent action bias from the latent endpoint mismatch. The controller uses images and latent states; joint angles are used only for evaluation. Because the model does not expose the local linear dynamics required by LQR, Tube-MPPI and RMPPI use MPPI as their auxiliary controller.

The pretrained LeWM model has prediction limitations that can prevent reliable task completion even without an additional action disturbance. To evaluate compensation for

the added disturbance on tasks that the nominal model and planner can already solve, we select 20 tasks for which disturbance-free MPPI has a final-20-step mean error no greater than 0.025 rad, then evaluate every disturbed method on the same tasks and planner seeds. Success requires this tail error to be below 0.05 rad. As shown in Figure. 9 and Table 4, MPPI+SDOB succeeds in 19 trials and lowers the mean tail error to 0.030 rad, versus 0.427, 0.185, and 0.176 rad for MPPI, Tube-MPPI, and RMPPI. Within this nominally solvable task subset, SDOB recovered successful performance in 19 of 20 trials under the imposed action bias.

4) *Rocket Collect cost-disturbance task:* This study evaluates the complete cost-compensated controller under faster obstacle motion, using reconstructed cost maps to augment world-model planning scores. The rocket must

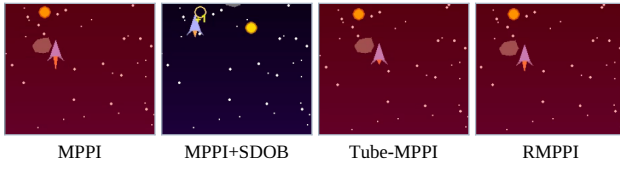


Figure 10. Final Rocket collect observations from representative evaluations, where MPPI+SDOB reduces collisions and improves task success.

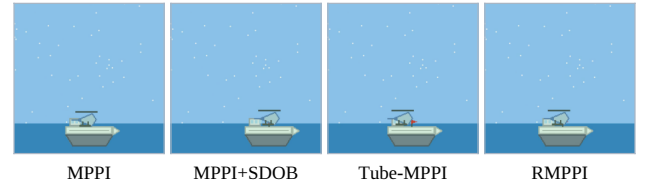


Figure 11. Final Landing observations from representative evaluations, where MPPI+SDOB achieves a smaller touchdown error.

Table 5. MMBench2 simulation results over 20 seeds. Touchdown center errors use trials that reach the deck.

Method	Rocket Collect				Visual Landing			
	Success	Collision	Control TV	P95 planning time (s)	Success	Touchdown center error (px)	Control TV	P95 planning time (s)
MPPI	16/20	4/20	0.409 ± 0.071	2.116 ± 0.093	20/20	20.49 ± 10.60	0.955 ± 0.144	12.131 ± 1.057
MPPI+SDOB	19/20	0/20	0.421 ± 0.071	2.219 ± 0.201	20/20	7.38 ± 6.90	0.835 ± 0.203	12.454 ± 1.072
Tube-MPPI	16/20	4/20	0.518 ± 0.071	2.491 ± 0.206	20/20	14.54 ± 4.70	0.764 ± 0.162	12.556 ± 0.717
RMPPI	15/20	4/20	0.537 ± 0.130	2.310 ± 0.171	20/20	16.61 ± 4.60	0.795 ± 0.122	12.670 ± 0.986

collect a coin while avoiding moving asteroids. For control, the available feedback consists of the current RGB image and the rocket's position and velocity. The released MMBench2 (Hansen and Wang 2026) world model uses the image and candidate action sequences to predict rewards and collision-risk scores through its reward head. A nominal motion model predicts rocket positions along each candidate path, and the coin detected in the image supplies the goal. The controller combines the learned scores with the distance to the coin and penalties on control effort and variation. For SDOB, the decoder reconstructs each current image, from which asteroid pixels are extracted to define an obstacle-distance cost map. The cost observer needs two successive maps queried at the same 17×17 grid points to estimate how the cost pattern moves. The controller then shifts the current map according to this estimate and adds a penalty where the predicted motion raises the cost along a candidate path. Thus, the reward head supplies the basic planning scores, while the reconstructed cost maps supply the SDOB correction. We increase asteroid speed to 1.5 times its nominal value and replan every two environment steps. MPPI, MPPI+SDOB, Tube-MPPI, and RMPPI are tested on the same 20 random seeds with the same world model and rollout budget. Success requires collecting the coin without collision within 140 steps. The controller has no access to simulator-provided obstacle states or collision geometry.

Figure 10 and Table 5 show that MPPI+SDOB succeeds in 19/20 trials with no collisions, compared with 16/20 for MPPI and Tube-MPPI and 15/20 for RMPPI. Its runs have slightly higher control variation than MPPI, while planning time remains comparable. For these evaluated seeds, the cost-compensated configuration produced fewer collisions under the specified increase in asteroid speed.

5) Visual Landing task: This study evaluates touchdown alignment when input bias and deck motion affect the landing task. The helicopter must land on a moving deck. For control, the inputs are the current RGB image and a fixed landing reference image. Helicopter and deck positions are detected in the current image, and successive helicopter positions provide a velocity estimate. The released MMBench2 world model scores candidate actions using predicted rewards and similarity to the reference image in its feature space. A nominal motion model also predicts helicopter positions and velocities, allowing the controller to evaluate distance to the detected deck and touchdown safety. SDOB uses two sets of feedback. The state observer takes recent image-based helicopter positions, the executed control inputs, and the nominal motion model to estimate an input bias that explains the observed motion. This estimate is included in both the world-model and motion-model predictions when evaluating new actions. The cost observer queries the landing cost defined by the detected deck at the same 11×11 grid points across successive observations. It estimates the motion of this cost map, which the controller uses to move its future landing targets. We add an unknown horizontal input bias of 0.16 and increase deck speed to 1.6 times nominal. The same four methods are tested on the same 20 random seeds with a common world model, replanning every four environment steps within a 96-step limit. First deck contact counts as success; touchdown center error measures landing accuracy. Simulator state and deck velocity are used only for evaluation.

As shown in Figure 11 and Table 5, all methods reach the deck in 20/20 trials. MPPI+SDOB reduces mean touchdown center error to 7.38 pixels, compared with 20.49 for MPPI, 14.54 for Tube-MPPI, and 16.61 for RMPPI. Tube-MPPI has



Figure 12. Representative real-car navigation snapshots, where MPPI+SDOB achieves faster collision-free navigation.

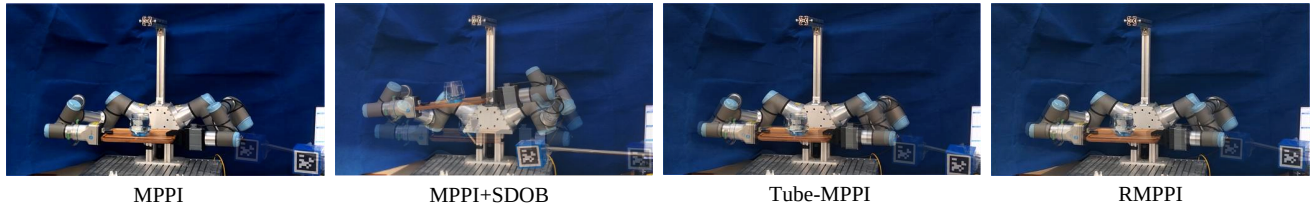


Figure 13. Representative dual-arm obstacle-avoidance snapshots, where SDOB responds earlier to the moving obstacle and successfully avoids collision.

Table 6. Completion times in the representative real-car runs.

Method	Completion time (s)
MPPI	12.13
MPPI+DOB	14.04
MPPI+SDOB	9.29
Tube-MPPI	12.07
RMPPI	13.10

lower control variation, indicating that SDOB’s main benefit in this task is improved touchdown alignment.

Remark 3. In the world-model simulations, planning is already slow without disturbance estimation: baseline MPPI has mean P95 planning times of 0.46 s, 2.12 s, and 12.13 s for Reacher, Rocket Collect, and Landing, respectively. These exceed the corresponding simulation-time control intervals, and disturbance estimation adds further computation. In these three tasks, the simulator is therefore paused during planning, including disturbance estimation, and advances only after computation finishes. Environment-step durations and replanning intervals refer to simulation time; reported planning times measure wall-clock computation. Reducing the combined computation time of planning and disturbance estimation is therefore an urgent priority for real-time feedback control with world models.

5.2 Experimental Results

Three robot studies illustrate SDOB with different prediction interfaces. Vehicle and dual-arm obstacle avoidance provide representative closed-loop demonstrations with state-space models. In the PointWorld (Huang et al. 2026) box-lifting study, SDOB uses initial feedback to correct model predictions for planning. The resulting plan is then executed

without replanning, and task performance is compared over 20 paired trials.

1) *Real-car obstacle-avoidance navigation task:* Vehicle motion is compared with nominal rollouts, while a moving person changes the perceived cost field. No separate person detection is performed; SDOB estimates the resulting cost-field motion from temporal changes in the cost queries. No ground-truth disturbance label is available, so the trial is assessed by route choice, collision avoidance, and completion time. In the representative runs in Figure 12, MPPI+SDOB follows the shortest route and finishes in 9.29 s, as listed in Table 6.

2) *Dual-arm obstacle-avoidance task:* The minimum collision distance is set to 0 cm, so no additional safety margin is imposed. The obstacle carries an AprilTag (Olson 2011) to update its current pose in the collision model, but no independent measurement of its Cartesian velocity or future trajectory is provided to the controller. SDOB instead estimates the obstacle motion from temporal changes in the collision-cost queries. In the representative trials in Figure 13, MPPI+SDOB begins avoidance earlier and avoids the obstacle, while the shown comparison runs collide.

3) *PointWorld box lifting with initial feedback calibration:* A dual-arm robot lifts a grasped box by 300 mm using a frozen PointWorld model. This experiment examines feedback calibration for planning with the frozen predictor. Forward inference is too slow for real-time replanning in the current implementation, so the study compares MPPI and MPPI+SDOB using a single plan, with no replanning or observer updates during execution. An initial 50 mm lift provides RGB-D feedback to calibrate the state-effect

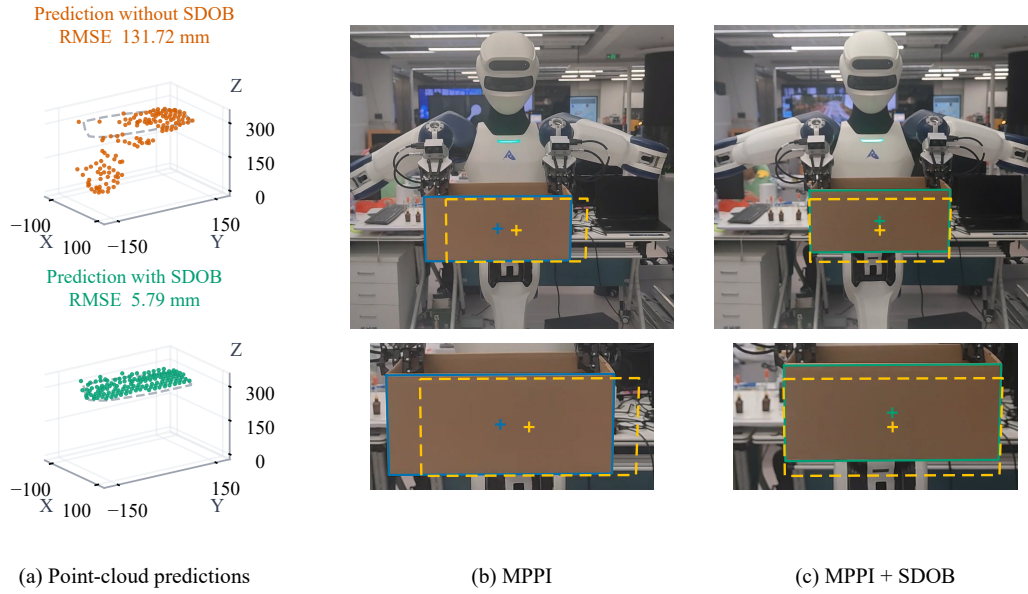


Figure 14. PointWorld box-lifting results. (a) Point-cloud motion predictions without and with SDOB. (b) Real-robot results with MPPI. (c) Real-robot results with MPPI+SDOB, showing a smaller box-lifting error.

Table 7. PointWorld box-lifting results over 20 paired trials.

Metric	MPPI	MPPI+SDOB
Final task RMSE (mm)	55.26 ± 14.99	22.61 ± 12.58
Final lift (mm)	287.39 ± 24.91	306.60 ± 12.73

correction. Alongside the PointWorld prediction, a rigid-body prediction assumes that the grasped box follows the translation and rotation of the two hands. This rigid-body prediction specifies the task-specific correction prior. SDOB samples a scalar correction coefficient between zero and one and weights the candidates according to how closely their corrected predictions match the observed box motion during this initial lift. For each subsequent candidate action, the estimated coefficient scales the difference between its rigid-body and PointWorld predictions. This correction is added to the predicted box points at each horizon step. The estimated coefficient remains fixed, while the correction varies with the candidate hand trajectory. MPPI then evaluates the corrected box motion against the lifting target to select an action sequence, with the PointWorld model weights kept frozen. As shown in Figure 14, the comparison includes point-cloud motion predictions and representative real-robot lifting results. The results over 20 trials are summarized in Table 7. The feedback-calibrated configuration achieved a lower mean final task RMSE than the uncorrected PointWorld planner over these trials.

6 Conclusion and Discussion

This paper presented SDOB for disturbance estimation and compensation through state-rollout and cost-query

interfaces. The state channel estimates corrections to predicted motion, while the cost channel estimates local changes in the cost field. Simulation and robot studies demonstrated improved task performance across analytical models, simulators, and learned world models.

Current world models have two limitations for disturbance-aware control. First, planning with these models requires repeated forward evaluations and already takes too long for high-frequency closed-loop control. Additional model evaluations for disturbance estimation further increase this computational burden. Second, their latent representations often lack a clear physical interpretation, making it difficult to represent disturbances and relate latent prediction errors to physical effects. Our world-model demonstrations are therefore limited to relatively simple tasks with task-specific disturbance representations. Future work will explore more efficient joint planning and disturbance estimation, as well as models trained with explicit disturbance inputs to support compensation in more complex tasks.

Code Availability

The code release includes source code, configurations, and simulation scripts for mobile-robot navigation, the inverted pendulum, and LeWM Reacher compensation, available at <https://github.com/sampling-based-dob/sampling-based-dob>.

References

- Abraham I, Handa A, Ratliff N, Lowrey K, Murphey TD and Fox D (2020) Model-based generalization under parameter uncertainty using path integral control. *IEEE Robotics and Automation Letters* 5(2): 2864–2871. <https://doi.org/10.1109/LRA.2020.2972836>.
- Agro B, Sykora Q, Casas S and Urtasun R (2023) Implicit occupancy flow fields for perception and prediction in self-driving. In: *2023 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. pp. 1379–1388. <https://doi.org/10.1109/CVPR52729.2023.00139>.
- Ai B, Tian S, Shi H, Wang Y, Pfaff T, Tan C, Christensen HI, Su H, Wu J and Li Y (2025) A review of learning-based dynamics models for robotic manipulation. *Science Robotics* 10(106): adt1497. <https://doi.org/10.1126/scirobotics.adt1497>.
- Bardes A, Garrido Q, Ponce J, Chen X, Rabbat M, LeCun Y, Assran M and Ballas N (2024) Revisiting feature prediction for learning visual representations from video. <https://doi.org/10.48550/arXiv.2404.08471>.
- Biswas S, Casas S, Sykora Q, Agro B, Sadat A and Urtasun R (2024) QuAD: Query-based interpretable neural motion planning for autonomous driving. In: *2024 IEEE International Conference on Robotics and Automation (ICRA)*. pp. 14236–14243. <https://doi.org/10.1109/ICRA57147.2024.10610648>.
- Carius J, Ranftl R, Farshidian F and Hutter M (2022) Constrained stochastic optimal control with learned importance sampling: A path integral approach. *The International Journal of Robotics Research* 41(2): 189–209. <https://doi.org/10.1177/02783649211047890>.
- Chebotaev Y, Handa A, Makoviychuk V, Macklin M, Issac J, Ratliff N and Fox D (2019) Closing the sim-to-real loop: Adapting simulation randomization with real world experience. In: *2019 International Conference on Robotics and Automation (ICRA)*. pp. 8973–8979. <https://doi.org/10.1109/ICRA.2019.8793789>.
- Chen WH, Ballance DJ, Gawthrop PJ and O'Reilly J (2000a) A nonlinear disturbance observer for robotic manipulators. *IEEE Transactions on Industrial Electronics* 47(4): 932–938. <https://doi.org/10.1109/41.857974>.
- Chen WH, Yang J, Guo L and Li S (2016) Disturbance-observer-based control and related methods—an overview. *IEEE Transactions on Industrial Electronics* 63(2): 1083–1095. <https://doi.org/10.1109/TIE.2015.2478397>.
- Chen XK, Komada S and Fukuda T (2000b) Design of a nonlinear disturbance observer. *IEEE Transactions on Industrial Electronics* 47(2): 429–437. <https://doi.org/10.1109/41.836359>.
- Gandhi MS, Vlahov B, Gibson J, Williams G and Theodorou EA (2021) Robust model predictive path integral control: Analysis and performance guarantees. *IEEE Robotics and Automation Letters* 6(2): 1423–1430. <https://doi.org/10.1109/LRA.2021.3057563>.
- Gao Z (2003) Scaling and bandwidth-parameterization based controller tuning. In: *Proceedings of the 2003 American Control Conference*, volume 6. pp. 4989–4996. <https://doi.org/10.1109/ACC.2003.1242516>.
- Garcia CE, Prett DM and Morari M (1989) Model predictive control: Theory and practice—a survey. *Automatica* 25(3): 335–348. [https://doi.org/10.1016/0005-1098\(89\)90002-2](https://doi.org/10.1016/0005-1098(89)90002-2).
- Ha D and Schmidhuber J (2018) Recurrent world models facilitate policy evolution. In: *Advances in Neural Information Processing Systems*.
- Hafner D, Lillicrap T, Ba J and Norouzi M (2020) Dream to control: Learning behaviors by latent imagination. In: *International Conference on Learning Representations (ICLR)*. URL <https://openreview.net/forum?id=S110TC4tDS>.
- Hafner D, Lillicrap T, Fischer I, Villegas R, Ha D, Lee H and Davidson J (2019) Learning latent dynamics for planning from pixels. In: *Proceedings of the 36th International Conference on Machine Learning, Proceedings of Machine Learning Research*, volume 97. pp. 2555–2565.
- Hafner D, Pasukonis J, Ba J and Lillicrap T (2025) Mastering diverse control tasks through world models. *Nature* 640(8059): 647–653. <https://doi.org/10.1038/s41586-025-08744-2>.
- Han J (2009) From PID to active disturbance rejection control. *IEEE Transactions on Industrial Electronics* 56(3): 900–906. <https://doi.org/10.1109/TIE.2008.2011621>.
- Hansen N, Su H and Wang X (2024) TD-MPC2: Scalable, robust world models for continuous control. In: *International Conference on Learning Representations (ICLR)*.
- Hansen N and Wang X (2026) Hallucination in world models is predictable and preventable. *arXiv preprint* <https://doi.org/10.48550/arXiv.2606.27326>.
- Horn BKP and Schunck BG (1981) Determining optical flow. *Artificial Intelligence* 17(1–3): 185–203. [https://doi.org/10.1016/0004-3702\(81\)90024-2](https://doi.org/10.1016/0004-3702(81)90024-2).
- Hornung A, Wurm KM, Bennewitz M, Stachniss C and Burgard W (2013) OctoMap: An efficient probabilistic 3D mapping framework based on octrees. *Autonomous Robots* 34(3): 189–206. <https://doi.org/10.1007/s10514-012-9321-0>.
- Huang W, Chao YW, Mousavian A, Liu MY, Fox D, Mo K and Li FF (2026) PointWorld: Scaling 3D world models for in-the-wild robotic manipulation. <https://doi.org/10.48550/arXiv.2601.03782>.
- Jaquier N, Welle MC, Gams A, Yao K, Fichera B, Billard A, Ude A, Asfour T and Kragić D (2025) Transfer learning in robotics: An

- upcoming breakthrough? a review of promises and challenges. *The International Journal of Robotics Research* 44(3): 465–485. <https://doi.org/10.1177/02783649241273565>.
- Kadian A, Truong J, Gokaslan A, Clegg A, Wijmans E, Lee S, Savva M, Chernova S and Batra D (2020) Sim2Real predictivity: Does evaluation in simulation predict real-world performance? *IEEE Robotics and Automation Letters* 5(4): 6670–6677. <https://doi.org/10.1109/LRA.2020.3013848>.
- Kang L and Joseph VR (2016) Kernel approximation: From regression to interpolation. *SIAM/ASA Journal on Uncertainty Quantification* 4(1): 112–129. <https://doi.org/10.1137/130948112>.
- Kobilarov M (2012) Cross-entropy motion planning. *The International Journal of Robotics Research* 31(7): 855–871. <https://doi.org/10.1177/0278364912444543>.
- Koptev M, Figueroa N and Billard A (2024) Reactive collision-free motion generation in joint space via dynamical systems and sampling-based MPC. *The International Journal of Robotics Research* 43(13): 2049–2069. <https://doi.org/10.1177/02783649241246557>.
- LeCun Y (2022) A path towards autonomous machine intelligence. OpenReview position paper.
- Maes L, Le Lidec Q, Scieur D, LeCun Y and Balestrieri R (2026) LeWorldModel: Stable end-to-end joint-embedding predictive architecture from pixels. *arXiv preprint* <https://doi.org/10.48550/arXiv.2603.19312>.
- Mahjourian R, Kim J, Chai Y, Tan M, Sapp B and Angelov D (2022) Occupancy flow fields for motion forecasting in autonomous driving. *IEEE Robotics and Automation Letters* 7(2): 5639–5646. <https://doi.org/10.1109/LRA.2022.3151613>.
- Mayne DQ, Rawlings JB, Rao CV and Sokaert POM (2000) Constrained model predictive control: Stability and optimality. *Automatica* 36(6): 789–814. [https://doi.org/10.1016/S0005-1098\(99\)00214-9](https://doi.org/10.1016/S0005-1098(99)00214-9).
- Mohammadi A, Tavakoli M, Marquez HJ and Hashemzadeh F (2013) Nonlinear disturbance observer design for robotic manipulators. *Control Engineering Practice* 21(3): 253–267. <https://doi.org/10.1016/j.conengprac.2012.10.008>.
- Nadaraya EA (1964) On estimating regression. *Theory of Probability and Its Applications* 9(1): 141–142. <https://doi.org/10.1137/1109020>.
- Nuss D, Reuter S, Thom M, Yuan T, Krehl G, Maile M, Gern A and Dietmayer K (2018) A random finite set approach for dynamic occupancy grid maps with real-time application. *The International Journal of Robotics Research* 37(8): 841–866. <https://doi.org/10.1177/0278364918775523>.
- Ohnishi K, Shibata M and Murakami T (1996) Motion control for advanced mechatronics. *IEEE/ASME Transactions on Mechatronics* 1(1): 56–67. <https://doi.org/10.1109/3516.491410>.
- Oleynikova H, Taylor Z, Fehr M, Siegwart R and Nieto J (2017) Voxblox: Incremental 3D euclidean signed distance fields for on-board MAV planning. In: *2017 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. pp. 1366–1373. <https://doi.org/10.1109/IROS.2017.8202315>.
- Olson E (2011) AprilTag: A robust and flexible visual fiducial system. In: *2011 IEEE International Conference on Robotics and Automation*. pp. 3400–3407. <https://doi.org/10.1109/ICRA.2011.5979561>.
- OpenAI, Andrychowicz M, Baker B, Chociej M, Józefowicz R, McGrew B, Pachocki J, Petron A, Plappert M, Powell G, Ray A, Schneider J, Sidor S, Tobin J, Welinder P, Weng L and Zaremba W (2020) Learning dexterous in-hand manipulation. *The International Journal of Robotics Research* 39(1): 3–20. <https://doi.org/10.1177/0278364919887447>.
- Ostafew CJ, Schoellig AP and Barfoot TD (2016) Robust constrained learning-based NMPC enabling reliable mobile robot path tracking. *The International Journal of Robotics Research* 35(13): 1547–1563. <https://doi.org/10.1177/0278364916645661>.
- Park G and Back J (2024) Survey of disturbance observer-based controller and related challenging problems. *Journal of Institute of Control, Robotics and Systems* 30(4): 363–372. <https://doi.org/10.5302/J.ICROS.2024.24.0045>.
- Pinneri C, Sawant S, Blaes S, Achterhold J, Stueckler J, Rolinek M and Martius G (2021) Sample-efficient cross-entropy method for real-time planning. In: *Proceedings of the 2020 Conference on Robot Learning, Proceedings of Machine Learning Research*, volume 155. pp. 1049–1065.
- Rubinstein RY (1997) Optimization of computer simulation models with rare events. *European Journal of Operational Research* 99(1): 89–112. [https://doi.org/10.1016/S0377-2217\(96\)00385-2](https://doi.org/10.1016/S0377-2217(96)00385-2).
- Sariyildiz E, Oboe R and Ohnishi K (2020) Disturbance observer-based robust control and its applications: 35th anniversary overview. *IEEE Transactions on Industrial Electronics* 67(3): 2042–2053. <https://doi.org/10.1109/TIE.2019.2903752>.
- Shim H and Jo NH (2009) An almost necessary and sufficient condition for robust stability of closed-loop systems with disturbance observer. *Automatica* 45(1): 296–299. <https://doi.org/10.1016/j.automatica.2008.10.009>.
- Tan J, Zhang T, Coumans E, Iscen A, Bai Y, Hafner D, Bohez S and Vanhoucke V (2018) Sim-to-real: Learning agile locomotion for quadruped robots. In: *Proceedings of Robotics: Science and Systems*. Pittsburgh, Pennsylvania. <https://doi.org/10.15607/RSS.2018.XIV.010>.

- Theodorou E, Buchli J and Schaal S (2010) A generalized path integral control approach to reinforcement learning. *Journal of Machine Learning Research* 11: 3137–3181.
- Tobin J, Fong R, Ray A, Schneider J, Zaremba W and Abbeel P (2017) Domain randomization for transferring deep neural networks from simulation to the real world. In: *2017 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. pp. 23–30. <https://doi.org/10.1109/IROS.2017.8202133>.
- Wang G, Ren K, Morgan AS and Hang K (2026) ManiDreams: An open-source library for robust object manipulation via uncertainty-aware task-specific intuitive physics. <https://doi.org/10.48550/arXiv.2603.18336>.
- Williams G, Aldrich A and Theodorou EA (2015) Model predictive path integral control using covariance variable importance sampling. <https://doi.org/10.48550/arXiv.1509.01149>.
- Williams G, Drews P, Goldfain B, Rehg JM and Theodorou EA (2018a) Information-theoretic model predictive control: Theory and applications to autonomous driving. *IEEE Transactions on Robotics* 34(6): 1603–1622. <https://doi.org/10.1109/TRO.2018.2865891>.
- Williams G, Goldfain B, Drews P, Saigol K, Rehg JM and Theodorou EA (2018b) Robust sampling based model predictive control with sparse objective information. In: *Proceedings of Robotics: Science and Systems*. Pittsburgh, Pennsylvania. <https://doi.org/10.15607/RSS.2018.XIV.042>.
- Yang J, Zheng WX, Li S, Wu B and Cheng M (2015) Design of a prediction-accuracy-enhanced continuous-time MPC for disturbed systems via a disturbance observer. *IEEE Transactions on Industrial Electronics* 62(9): 5807–5816. <https://doi.org/10.1109/TIE.2015.2450736>.
- Yin J, Dawson C, Fan C and Tsiotras P (2023) Shield model predictive path integral: A computationally efficient robust MPC method using control barrier functions. *IEEE Robotics and Automation Letters* 8(11): 7106–7113. <https://doi.org/10.1109/LRA.2023.3315211>.
- Yoneda T, Yang G, Walter MR and Stadie BC (2022) Invariance through latent alignment. In: *Proceedings of Robotics: Science and Systems*. New York City, NY, USA. <https://doi.org/10.15607/RSS.2022.XVIII.064>.
- Zeng W, Luo W, Suo S, Sadat A, Yang B, Casas S and Urtasun R (2019) End-to-end interpretable neural motion planner. In: *2019 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. pp. 8652–8661. <https://doi.org/10.1109/CVPR.2019.00886>.
- Zucker M, Ratliff N, Dragan AD, Pivtoraiko M, Klingensmith M, Dellin CM, Bagnell JA and Srinivasa SS (2013) CHOMP: Covariant hamiltonian optimization for motion planning. *The International Journal of Robotics Research* 32(9–10): 1164–1193. <https://doi.org/10.1177/0278364913488805>.